AMT Horizon

Functional Specification / Technical Specification Document

DOCUMENT VERSION 1.1

08/01/2026

Author

Name	Role
Alexis SANTOS	Program Manager / Tech Lead

Revision History

Date	Version	Description of Changes
04/01/2026	1.0	• Document creation and drafting of sections Overview, A to E. • Added Functional Requirements.
08/01/2026	1.1	• Translated code and technical sections to English. • Adapted for NextJS, Vitest, Neon, Drizzle, better-auth.

▶ Table of Contents

- I. General Overview
  - A. Product Description
  - B. Product Functional Capabilities
    - User Management
    - Ride Management
    - Admin View (Excel-like)
    - Billing
    - Notifications
    - Additional Features
  - C. Project Organization
    - Project Representatives
    - Stakeholders
- II. Requirements
  - A. Functional Requirements
    - User Management
    - Gestion des Courses
    - Admin View (Excel-like)
    - Billing
    - Notifications
    - Additional Features

        - Security
    - B. Non-Functional Requirements
    - C. Database Structure (PostgreSQL/Neon)
        - Conceptual Schema
        - Explanations
    - D. SQL Query Examples
        - 1. List unassigned rides
        - 2. Generate an invoice
        - 3. Update assignment request status
        - 4. Retrieve user's unread notifications
        - 5. List available drivers within a 5 km radius
        - 6. Calculate driver's average rating
        - 7. Send invoice via application
        - 8. List invoices eligible for reminder (unpaid, overdue)
        - 9. Send manual reminder
- Glossary

I. General Overview

Our client requested the development of a web/mobile application in NextJS with better-auth to manage taxi rides, their distribution, and billing. The goal is to provide an intuitive solution for admins, drivers, and customers, with advanced features such as data import via photo (OCR) and automatic ride suggestions.

A. Product Description

This application allows you to:

- **Cataloging** taxi rides.
- **Distributing** rides among drivers via a dynamic schedule.
- **Generating** invoices from rides (individual or grouped).
- **Suggesting** rides to drivers and allowing them to request assignment.
- **Importing** ride data via form, text, schema, or photo (OCR).

B. Product Functional Capabilities

User Management

- **Registration/Login**: Distinct roles (Admin, Driver, Customer) with secure authentication (better-Auth).
- **Password Recovery**: Reset link valid for 24 hours.

Ride Management

- **Manual Addition**: Form to enter details (date, time, location, price).
- **Import via Text/Photo**: Automatic data extraction via OCR (Tesseract.js/Google Vision).
- **Ride Suggestions**: Matching algorithm based on availability and location.
- **Assignment Request**: Drivers can request to be assigned to a ride.

Admin View (Excel-like)

- **Dynamic Table**: Filters, sorting, data export.
- **Export**: Generate CSV/PDF files.

Billing

- **Invoice Creation**: Select rides, automatic numbering.
- **Invoice Sending**: By email, with status tracking.

Notifications

- **Real-time Alerts**: New rides, assignments, invoices.

Additional Features

- **Geolocation**: Integration with Mapbox/Google Maps.
- **Online Payment**: Stripe/PayPal.
- **Internal Chat**: Communication between drivers and customers.

C. Project Organization

Project Representatives

Project Owner	Represented by...
Client	Alexis SANTOS (Program Manager)

Stakeholders

Stakeholder	Interest
Admins	Full management of rides and invoices.
Drivers	View suggested rides, request assignment.
Customers	Add rides, track trips.

II. Requirements

A. Functional Requirements

User Management

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-001	Registration/Login	All	Email, password, role	Account created, active session	Roles: Admin, Driver, Customer

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-002	Password Recovery	All	Email	Reset link	Link valid for 24h
FR-003	Driver Account Validation	Admin	ID card, driver's license	Account validated or rejected	Manual verification within 24h.
FR-004	Profile Modification	All	New email, phone number	Profile updated, notification	Unique email, valid format.
FR-005	Account Deactivation	Admin	User, Reason (optional)	Account deactivated	Account disabled, notification, Data archiving (GDPR).

Gestion des Courses

ID	Fonctionnalité	Acteurs	Entrées	Sorties	Règles Métier
FR-010	Manual Ride Addition	All	Form (date, time, location, price)	Ride recorded, notification	Required fields validated
FR-011	Ride Import via Photo	Admin, Drivers	Photo (ticket, invoice)	Extracted data, ride created	OCR + manual validation
FR-012	Ride Suggestions	System	Availability, location	List of suggested rides	Matching algorithm
FR-013	Manual Validation of OCR Imports	Admin	Extracted data, manual correction	Ride validated or rejected	Notification required for rejection (reason mandatory).
FR-014	Ride History	Drivers, Customers	Filters (date, status)	List of past rides	Customers only see their own rides.
FR-015	Ride Cancellation	Drivers, Customers	Reason, ride ID	Ride cancelled, notification	Max delay: 2 hours before departure.
FR-016	Driver Rating	Customers	Rating (1-5), comment	Rating recorded, average updated	Only after a completed ride.
FR-017	Automatic Ride Suggestions	System	Location, availability	List of 3 priority rides	Algorithm based on proximity and history.

ID	Fonctionnalité	Acteurs	Entrées	Sorties	Règles Métier
FR-018	Assignment Request with Comment	Drivers	Ride ID, message	Request sent, admin notification	Max 5 simultaneous requests.

Admin View (Excel-like)

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-020	Dynamic Table	Admin	Filters (date, status)	Sortable/exportable list	Access restricted to admins
FR-021	Custom Export	Admin	Selected columns	CSV/PDF file with filtered data	Limit: 10,000 rows per export.
FR-022	Ride Statistics	Admin	Period, data type	Charts (number, revenue, etc.)	Automatic daily refresh.

Billing

ID	Fonctionnalité	Acteurs	Entrées	Sorties	Règles Métier
FR-030	Invoice Creation	Admin	Ride selection	Generated PDF invoice	Automatic numbering
FR-031	Grouped Billing	Admin	Ride selection	Single PDF invoice	Grouping by customer or period.
FR-032	Automatic Reminder	System	Overdue due date	Reminder email	Max 3 reminders (D+7, D+14, D+30).
FR-033	Exceptional Discount	Admin	Invoice ID, percentage	Updated invoice	Justification required.
FR-034	Send Invoice via App	Admin	Invoice ID, recipient	Invoice sent (PDF attached), status updated	Immediate sending, history preserved. Notifies customer via email and in-app.
FR-035	Manual Reminder Management	Admin	Invoice ID, custom message	Reminder sent (email/app), status updated	Only after explicit admin validation. Max 3 reminders (configurable).

Notifications

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-040	Real-time Notifications	All	Event (ride, invoice)	Email/in-app alert	Channel customization
FR-041	Channel Customization	All	Preferences (email, app)	Channels updated	Email by default, can be disabled.

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-042	Urgent Notifications	System	Critical event	Push + email alert	Ex: ride cancellation by customer.

Additional Features

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-050	Route Geolocation	Drivers, Customers	GPS coordinates	Interactive map	Mapbox/Google Maps integration
FR-051	Real-time Chat	Drivers, Customers	Text message	Saved conversation	History kept for 30 days.
FR-052	Live GPS Tracking	Customers	Ride ID	Driver position in real-time	Can be disabled by driver.
FR-053	Partial Payment	Customers	Partial amount	Invoice updated	Minimum 30% of total amount.
FR-054	Calendar Integration	Drivers	Google Calendar/Outlook link	Synchronized events	Bidirectional synchronization.

Security

ID	Feature	Actors	Inputs	Outputs	Business Rules
FR-060	Activity Log	Admin	Filters (user, action)	List of actions	6-month retention (GDPR).
FR-061	Two-Factor Authentication	All	SMS/email code	Access granted or denied	Mandatory for admins.

B. Non-Functional Requirements

- **OCR**: Minimum 90% accuracy for data extraction.
- **Performance**: Response time < 2s for critical actions.
- **Security**: Encryption of sensitive data (GDPR).

C. Database Structure (PostgreSQL/Neon)

Conceptual Schema

```
-- Users Table
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
```

```
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) CHECK (role IN ('admin', 'driver', 'customer')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Rides Table
CREATE TABLE rides (
    ride_id SERIAL PRIMARY KEY,
    departure VARCHAR(255) NOT NULL,
    destination VARCHAR(255) NOT NULL,
    departure_time TIMESTAMP NOT NULL,
    arrival_time TIMESTAMP,
    price DECIMAL(10, 2) NOT NULL,
    status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending',
'assigned', 'completed', 'cancelled')),
    photo_url VARCHAR(512),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    user_id INT REFERENCES users(user_id),
    driver_id INT REFERENCES users(user_id)
);

-- Assignment Requests Table
CREATE TABLE assignment_requests (
    request_id SERIAL PRIMARY KEY,
    ride_id INT REFERENCES rides(ride_id),
    driver_id INT REFERENCES users(user_id),
    status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending',
'approved', 'rejected')),
    requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Invoices Table
CREATE TABLE invoices (
    invoice_id SERIAL PRIMARY KEY,
    ride_id INT REFERENCES rides(ride_id),
    amount DECIMAL(10, 2) NOT NULL,
    status VARCHAR(20) DEFAULT 'unpaid' CHECK (status IN ('unpaid',
'paid', 'cancelled')),
    invoice_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    due_date TIMESTAMP,
    pdf_url VARCHAR(512),
    sent_via_app BOOLEAN DEFAULT FALSE,
    app_sent_at TIMESTAMP,
    reminders_sent INT DEFAULT 0,
    last_reminder_message TEXT
);

-- Notifications Table
CREATE TABLE notifications (
    notification_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Ratings Table
CREATE TABLE ratings (
    rating_id SERIAL PRIMARY KEY,
    ride_id INT REFERENCES rides(ride_id),
    customer_id INT REFERENCES users(user_id),
    rating INT CHECK (rating BETWEEN 1 AND 5),
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Notification Preferences Table
CREATE TABLE notification_preferences (
    preference_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    email BOOLEAN DEFAULT TRUE,
    push BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Activity Logs Table
CREATE TABLE activity_logs (
    log_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    action VARCHAR(255) NOT NULL,
    details TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Explanations

- **users**: Stores user information (role, email, hashed password).
- **rides**: Details of trips, including times, status, and user links.
- **assignment_requests**: Manages driver assignment requests for rides.
- **invoices**: Invoices generated from rides.
- **notifications**: Alerts sent to users.

D. SQL Query Examples

1. List unassigned rides

```
SELECT * FROM rides
WHERE status = 'pending' AND driver_id IS NULL;
```

2. Generate an invoice

```
INSERT INTO invoices (ride_id, amount, status, invoice_date, due_date)
VALUES (1, 25.50, 'unpaid', NOW(), NOW() + INTERVAL '30 days');
```

3. Update assignment request status

```
UPDATE assignment_requests
SET status = 'approved'
WHERE request_id = 1;
```

4. Retrieve user's unread notifications

```
SELECT * FROM notifications
WHERE user_id = 1 AND is_read = FALSE;
```

5. List available drivers within a 5 km radius

```
-- Assuming `latitude` and `longitude` columns in the users table
SELECT u.user_id, u.email
FROM users u
WHERE u.role = 'driver'
  AND u.available = TRUE
  AND ST_DWithin(
      ST_MakePoint(u.longitude, u.latitude)::geography,
      ST_MakePoint(:client_longitude, :client_latitude)::geography,
      5000  -- 5 km in meters
  );
```

6. Calculate driver's average rating

```
SELECT AVG(rating) as average_rating
FROM ratings
WHERE driver_id = :driver_id;
```

7. Send invoice via application

```
UPDATE invoices
SET
    sent_via_app = TRUE,
    app_sent_at = NOW(),
    status = 'sent'
```

```
WHERE invoice_id = :invoice_id;
-- Send PDF by email and in-app notification (backend logic)
```

8. List invoices eligible for reminder (unpaid, overdue)

```
SELECT invoice_id, ride_id, amount, due_date
FROM invoices
WHERE status = 'unpaid'
  AND due_date < NOW()
  AND reminders_sent < 3;
```

9. Send manual reminder

```
UPDATE invoices
SET
    reminders_sent = reminders_sent + 1,
    last_reminder_message = :custom_message,
    status = 'reminder_sent'
WHERE invoice_id = :invoice_id;
-- Send email/notification with custom message
```

Glossary

- **OCR**: Optical Character Recognition, technology to extract text from images (tickets, invoices).
- **Neon**: Serverless service for PostgreSQL, optimized for scalable and modern applications.
- **GDPR**: General Data Protection Regulation, legal framework for personal data protection in Europe.
- **Stripe/PayPal**: Secure online payment solutions.
- **Mapbox/Google Maps**: Mapping APIs for geolocation and route display.
- **2FA**: Two-Factor Authentication, security method requiring two proofs of identity.
- **ST_DWithin**: PostgreSQL/PostGIS function for geospatial queries.